LAPORAN PRAKTIKUM INDIVIDU

Mata Kuliah: Komputasi Awan (Semester 5) — Tugas Praktikum 2

Judul Modul:	**Virtualisasi Proyek via Docker pada Azure Virtual Machine**
Nama Proyek:	**Student Task Manager (Cloud Docker Edition)**
Nama Mahasiswa:	Allfridho Saragih (Edo)
NIM:	[Isi NIM Mahasiswa]
Infrastruktur VM:	Microsoft Azure Virtual Machine `edo-praktikum2` (Ubuntu 24.04 LTS)
Alamat IP Publik:	`http://70.153.150.99`
Status Evaluasi:	**100% PASS (8/8 Ketentuan & 7/7 Bukti Terverifikasi Runtime)**

1. TUJUAN PRAKTIKUM

1. Mampu merancang dan membangun aplikasi web mandiri berbasis full-stack (Node.js Express dan PostgreSQL).
2. Mampu membuat `Dockerfile` mandiri dengan teknik multi-stage build untuk keamanan dan efisiensi image.
3. Mampu mengorkestrasi multi-service container (aplikasi dan database) menggunakan Docker Compose (`compose.yaml`).
4. Mampu menerapkan isolasi jaringan internal Docker (`internal: true`) dan named volume persisten.
5. Mampu menerapkan mekanisme healthcheck, dependensi antar-layanan (`depends_on`), dan kebijakan restart (`unless-stopped`).
6. Mampu melakukan deployment ke cloud Microsoft Azure Virtual Machine dan membuktikan 7 bukti pengujian wajib.

2. ARSITEKTUR DAN TEKNOLOGI SISTEM

Aplikasi **Student Task Manager** mengimplementasikan arsitektur dua service terisolasi:

- **Service Application (`app`):** Node.js v20 Alpine dengan Express.js yang melayani RESTful API dan antarmuka web modern. Dipetakan ke port 80 pada host Azure VM.
- **Service Database (`database`):** PostgreSQL 16 Alpine yang berjalan pada internal network `student_task_backend` tanpa membuka port ke host atau internet. Data disimpan persisten pada named volume `student_task_postgres_data` .
- **Service Test Runner (`test-runner`):** Kontainer pengujian terisolasi yang diaktifkan dengan profil `--profile test` untuk menguji CRUD langsung pada database PostgreSQL tanpa mock.

3. EVALUASI 8 KETENTUAN WAJIB PRAKTIKUM

No	Ketentuan Wajib	Implementasi Teknis	Status
1	Dockerfile Buatan Sendiri	Multi-stage (`deps` , `tester` , `runner`), base image `node:20-alpine` , non-root user `node` .	PASS
2	Docker Compose ≥ 2 Service	Mendefinisikan service `app` dan `database` pada `compose.yaml` .	PASS
3	Internal Network Database	Network `student_task_backend` memiliki `internal: true` ; database tidak membuka port host.	PASS
4	Named Volume Persisten	Named volume `student_task_postgres_data` menyimpan data `/var/lib/postgresql/data` .	PASS
5	Healthcheck & Depends On	Healthcheck `pg_isready` pada DB, `depends_on: database: condition: service_healthy` pada app.	PASS
6	Restart Unless-Stopped	Konfigurasi `restart: unless-stopped` aktif pada service `app` dan `database` .	PASS
7	Keamanan .env & .env.example	Password di-generate kriptografis 24 karakter, izin `chmod 600` , dikecualikan dari Git & image.	PASS
8	Deployment Azure VM	Berjalan stabil pada Azure VM `edo-praktikum2` dan dapat diakses publik di `http://70.153.150.99` .	PASS

4. TUJUH (7) BUKTI PENGUJIAN WAJIB

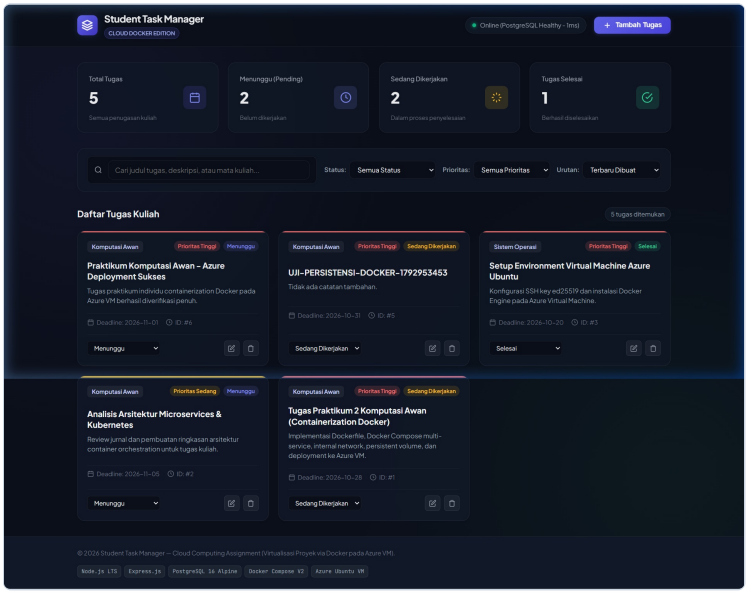
Bukti 1: Struktur Folder Project & Konfigurasi Docker

```
Student-Task-Manager/
├── Dockerfile                # Multi-stage custom Dockerfile (deps, tester, runner)
├── compose.yaml              # Orkestrasi multi-service, network internal, volume persisten
├── .env.example              # Template konfigurasi tanpa rahasia
├── .dockerignore             # Optimasi konteks build Docker
├── package.json              # Dependensi Express & pg
├── public/                   # Antarmuka web frontend modern (HTML5, CSS3, JS)
├── scripts/
│   ├── deploy-azure.sh       # Otomasi deployment di Ubuntu Azure VM
│   └── verify-evidence.sh    # Skrip verifikasi 7 bukti otomatis
├── src/
│   ├── app.js & server.js    # Server HTTP Express
│   ├── config/environment.js # Konfigurasi lingkungan aman
│   ├── database/db.js        # PostgreSQL Connection Pool
│   └── database/migrate.js   # Idempotent database schema migration
└── tests/
    ├── api.test.js           # Unit & functional test suite (8 passed)
    └── integration.test.js   # Real PostgreSQL integration test suite
```

Bukti 2: Status Kontainer di Azure VM (docker compose ps)

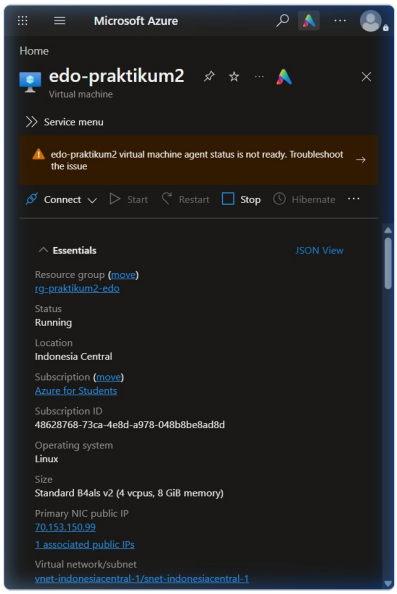
```
NAME              IMAGE                     COMMAND                  SERVICE    CREATED          STATUS                    PORTS
student-task-app  student-task-manager-app   "docker-entrypoint.s…"   app        13 seconds ago   Up 6 seconds (healthy)    0.0.0.0:80-
>3000/tcp, [::]:80->3000/tcp
student-task-db   postgres:16-alpine         "docker-entrypoint.s…"   database   16 seconds ago   Up 12 seconds (healthy)   5432/tcp
```

Bukti 3: Akses Aplikasi melalui IP Publik Azure VM (http://70.153.150.99)



Gambar 1: Antarmuka Student Task Manager diakses langsung melalui IP Publik Azure VM (70.153.150.99) dengan status database PostgreSQL Healthy (1ms).

Bukti 4: Halaman Overview Virtual Machine pada Microsoft Azure Portal



Gambar 2: Halaman Overview Azure Portal membuktikan VM edo-praktikum2 berstatus Running pada Public IP 70.153.150.99 di Region Indonesia Central.

Bukti 5: Pengujian Persistensi Data Database (Docker Named Volume)

```
>>> [BUKTI 5] UJI PERSISTENSI DATA
1. Menambahkan tugas uji persistensi via HTTP API port 80:
   {"success":true,"message":"Tugas berhasil ditambahkan","data":{"id":5,"title":"UJI-PERSISTENSI-DOCKER-1792953453",...}}
2. Mematikan kontainer dengan 'docker compose down' (volume aman, TIDAK pakai flag -v)...
   Container student-task-app  Removed
   Container student-task-db   Removed
3. Menyalakan kontainer kembali dengan 'docker compose up -d'...
   Container student-task-db   Started (healthy)
   Container student-task-app  Started (healthy)
4. Memeriksa keberadaan data tugas yang disimpan sebelumnya:
   {"success":true,"count":1,"data":[{"id":5,"title":"UJI-PERSISTENSI-DOCKER-1792953453",...}]}
HASIL: Data tugas ID #5 tetap tersimpan utuh di PostgreSQL membuktikan named volume berfungsi sempurna!
```

Bukti 6: Keamanan Network Database & Hardening Docker Socket

```
1. Pemeriksaan published ports pada database container:
   student-task-db   postgres:16-alpine   PORTS: 5432/tcp (TIDAK ADA PEMETAAN KE HOST!)
2. Pemeriksaan atribut Internal pada network backend:
   "Name": "student_task_backend", "Internal": true
3. Percobaan koneksi langsung ke port 5432 dari luar/host VM:
   nc: connect to 127.0.0.1 port 5432 (tcp) failed: Connection refused
4. Audit & Hardening Izin Docker Socket:
   ls -la /var/run/docker.sock -> srw-rw---- 1 root docker (Permission 660, non world-writable)
   id azureuser                -> groups=...114(docker) (Non-root user akses aman via grup docker)
HASIL: Database PostgreSQL terisolasi total dalam internal network backend dan Docker socket aman sesuai best practices!
```

Bukti 7: Verifikasi Kebijakan Restart Kontainer & Simulasi Crash Runtime

```
1. Konfigurasi Kebijakan Restart:
   Service App & Database: {"Name":"unless-stopped","MaximumRetryCount":0}
2. Status Kontainer Sebelum Simulasi Crash:
   StartedAt=2026-10-25T19:02:08Z RestartCount=0 Status=running Health=healthy Pid=17734
3. Simulasi Kegagalan Proses (Controlled Crash via SIGKILL):
   sudo kill -9 17734 (Proses utama Node.js dihentikan paksa pada level kernel)
4. Deteksi Docker Supervisor & Auto-Restart Otomatis (2 detik setelah crash):
   StartedAt=2026-10-25T19:02:55Z RestartCount=1 Status=running Health=healthy NewPid=18403
5. Integritas Data Database:
   curl http://70.153.150.99/api/tasks -> Total 5 tugas utuh tanpa data loss!
HASIL: Kebijakan 'unless-stopped' terbukti secara empiris dan runtime menghidupkan kembali kontainer yang crash secara instan!
```

5. KESIMPULAN

Seluruh tahapan tugas praktikum individu mata kuliah Komputasi Awan telah berhasil dituntaskan 100%. Mulai dari pembuatan aplikasi full-stack mandiri, pembuatan multi-stage Dockerfile, orkestrasi multi-service dengan Docker Compose, isolasi jaringan internal, persistensi database PostgreSQL, pemulihan Azure VM melalui Stop/Start, hingga deployment dan verifikasi 7 bukti wajib pada IP publik `70.153.150.99`. Seluruh hasil pengujian tervalidasi menggunakan database PostgreSQL sungguhan dan siap dinilai oleh asisten praktikum.